

Chapter 3

Data Link Layer –Part 1

Outcomes

- Upon completion of this chapter learner will be able to
 - Know and understand the various services offered by data link layer
 - Understand the sub layers of data link layer
 - Know the responsibilities of LLC or DLC and MAC sub layer
 - Understand framing ,framing errors , types of framing and various framing approaches.

Data Link Layer

- is second layer of OSI Layered Model.
- Also known as link layer.
- Is responsible for moving data (Frames) from one node to another— node-to-node communication
- This direct connection could be **point to point** or **broadcast**. Systems on broadcast network are said to be on same link.

- Data link layer is responsible for converting data stream to signals bit by bit and to send that over the underlying hardware. At the receiving end, Data link layer picks up data from hardware which are in the form of electrical signals, assembles them in a recognizable frame format, and hands over to upper layer.

DATA LINK LAYER FUNCTIONS (SERVICES)

- **Framing**
- **Physical Addressing**
- **Flow Control**
- **Error Control**
- **Access Control**

Framing

- The data link layer needs to pack bits in to frames, so that each frame is distinguishable from another.
- Our postal system practices a type of framing
- The simplest act of inserting a letter into an envelop separates one piece of information from other; the envelop serves as the delimiter.

Physical addressing

- A frame is the encapsulation of header and trailer information with the packet.
- In the header the source and the destination MAC address(hardware address) are dealt
- Hardware address is assumed to be unique on the link.
- It is encoded into hardware at the time of manufacturing.

Flow Control

- One of the duties of **data link control sub layer**
- The flow control in the data link layer is **end to end** flow control
- Stations on same link may have different speed or capacity.
- Speed matching mechanism
- coordinates the amount of data that can be sent before receiving device acknowledges .

Access control

- Media access control
- when host on the shared link tries to transfer the data, it has a high probability of collision.
- Data-link layer provides mechanism to equip capability of accessing a shared media among multiple Systems.

Synchronization

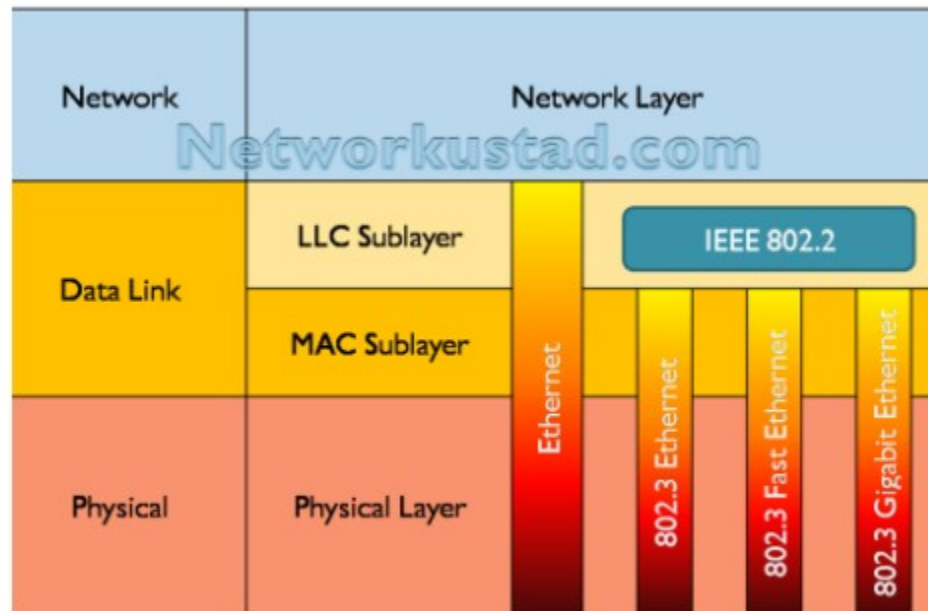
- When data frames are sent on the link, both machines must be synchronized in order to transfer to take place.

Error Control

- Sometimes signals may have encountered problem in transition and the bits are flipped.
- Approaches
 - Error correction
 - Error correction

- Data link layer has two sub-layers:
 - **Logical Link Control:** It deals with protocols, flow-control, and error control.
 - **Media Access Control:** It deals with actual control of media.

Data link sub layer



LLC or DLC

- Handles communication between the upper layer and lower layer
- Takes the network protocol data and adds control information to help deliver the packet to the destination.(flow control)

MAC sub layer

- Constitutes the lower sub layer of the data link layer.
- Implemented by hardware, typically in the computer NIC
- Two primary responsibilities
 - Data encapsulation
 - Media access control

- Data encapsulation
 - Frame assembly before transmission and frame disassembly upon reception of a frame
 - MAC layer adds a header and a trailer to the network layer PDU.
- Provides three primary functions
 - Framing
 - Physical addressing or MAC addressing
 - Error Control
 - Access control

- Responsible for the placement of frames on the media and the removal of frames from the media
- Directly communicate with the physical layer

Framing

- Framing | the data link layer separates a frame distinguishable from another frame
- frame=Header+ Network layer PDU + trailer.
- When node A wishes to transmit a frame to node b, it tells its adapter to transmit a frame from the nodes memory
- This results in a sequence of bits being sent over the link
- The adapter on node B then collects together the sequence of bits arriving on the link and deposits the corresponding frame in to B's memory.
- **Challenge:** what set of bits constitute a frame ?

Types of framing

- Fixed-size of the framing
- Variable size framing

- **Fixed-size framing**

- Here the size of the frame is fixed and so the frame length acts as a delimiter of the frame
- Consequently, it does not require additional boundary bits to identify the start and end of the frame.

- **Variable-size framing**

- Here the size of each frame to be transmitted may be different.
- So additional mechanism are kept to **mark** the end of the frame and the beginning of the next frame and this will help us to construct the frame without the framing error.

Framing approaches

- **Framing**

- **Bit oriented**

- It simply view the frame as a collection of bits
 - Data is transmitted as a sequence of bits that can be interpreted in the upper layer both as a text as well as multimedia.
 - Bit oriented protocol
 - HDLC-High-Level Data link Control

- **Byte oriented**

- One of the oldest approach to framing
- Here each frame is viewed as a collection of byte(characters) rather than bits.
- a.k.a character oriented approach
- Protocols
 - BISYNC-binary synchronous communication protocol
 - DDCMP– digital data communication message protocol
 - PPP-Point to point Protocol

- Clock based framing
 - The third approach to framing is the clock based framing
 - Example: SONET-synchronous Optical Network

FLOW CONTROL PROTOCOLS

Outcomes

- Upon completion of this chapter learner will be able to
 - Understand flow control
 - Know flow control protocols.
 - Understand the stop-and –wait protocol
 - Understand the stop-and –wait protocol ARQ
 - Understand the sliding window protocol (Go-back-N ARQ)

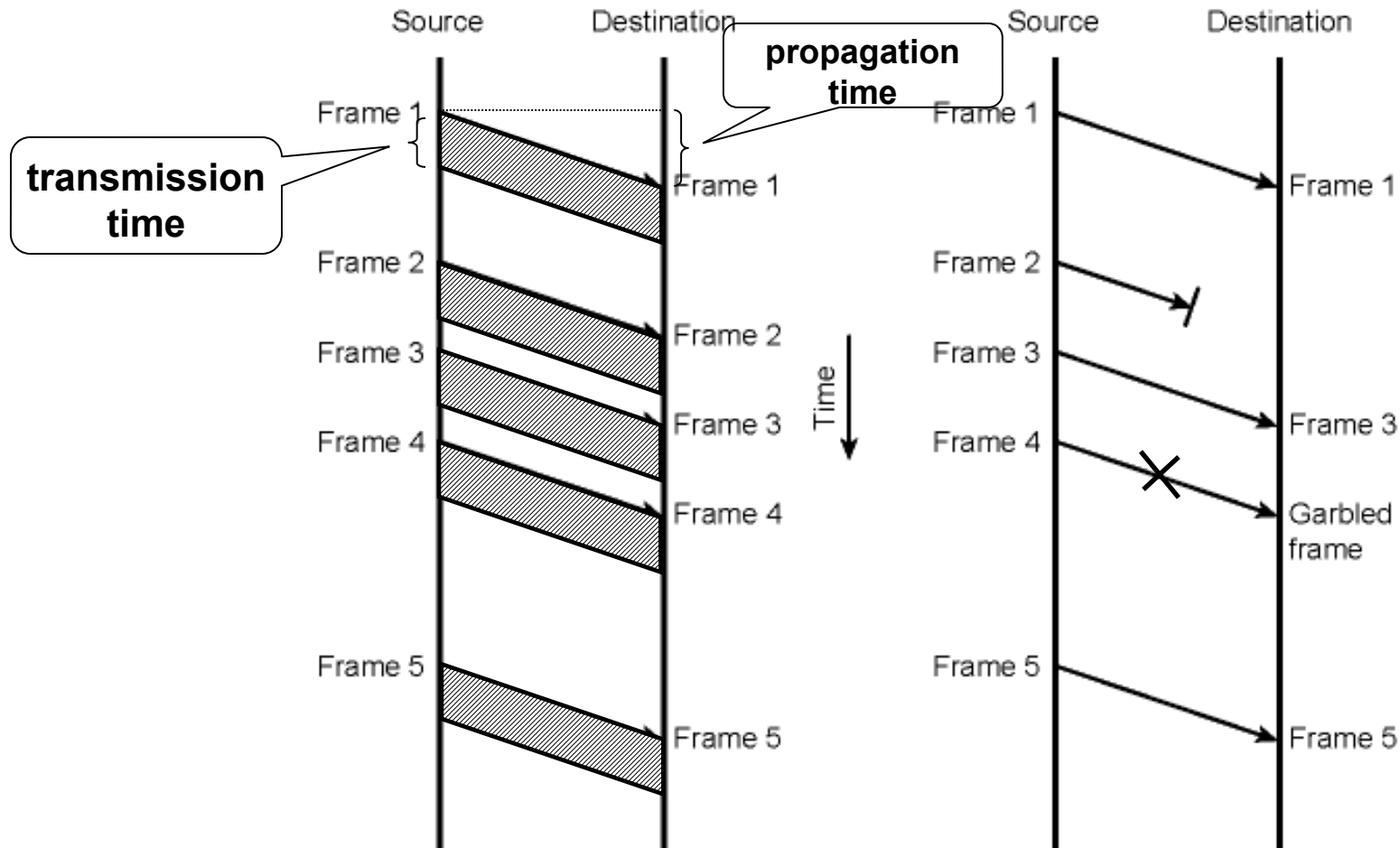
Flow Control

- Speed matching
- Flow control coordinates the amount of data that can be sent before receiving an acknowledgement
- Flow control is a set of procedures that tells the sender how much data it can transmit before it must wait for an acknowledgement from the receiver.
- Receiver has a limited speed at which it can process incoming data and a limited amount of memory in which to store incoming data
- The receiver must inform the sender before the limits are reached and request that the transmitter to send fewer frames or stop temporarily

Performance Metrics and Delays

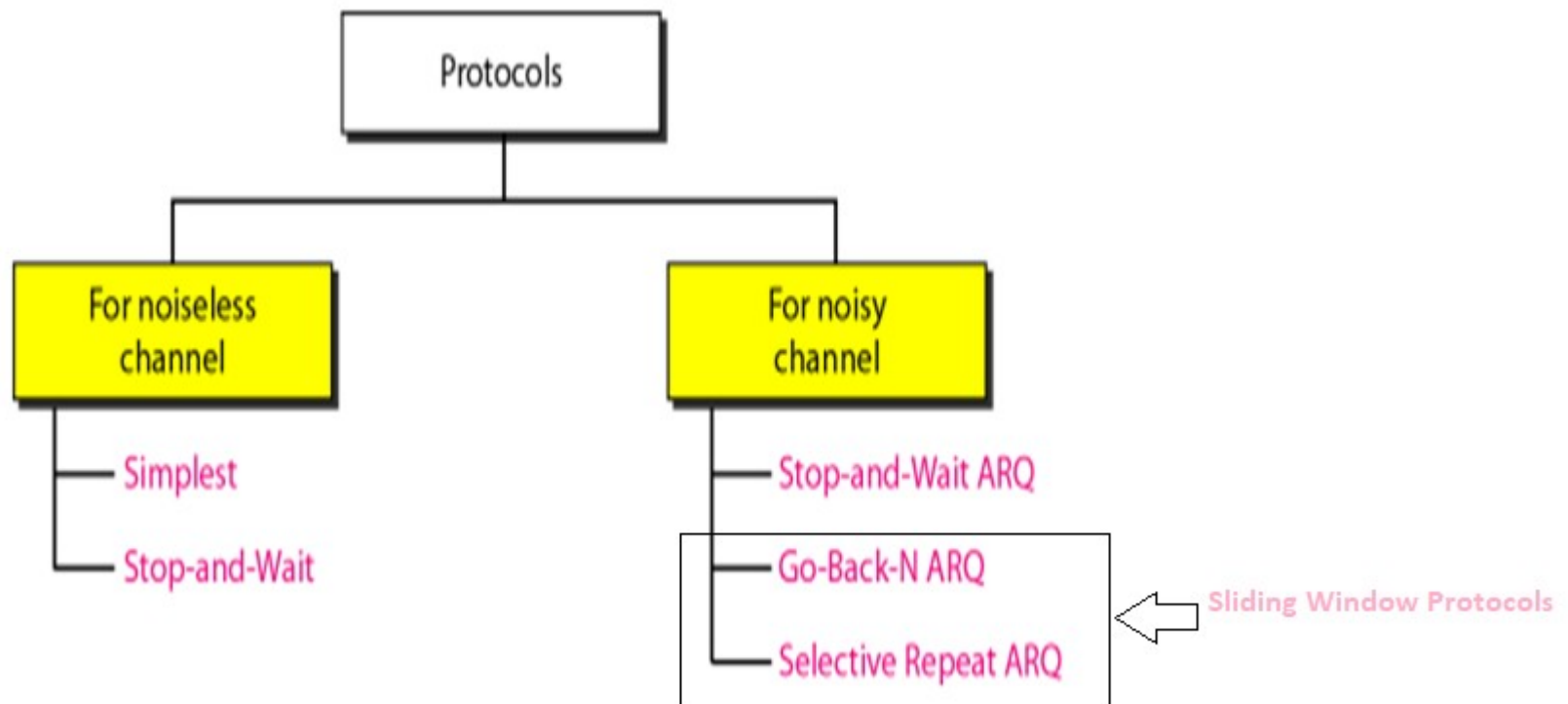
- Transmission time (delay)
 - Time taken to emit all bits into medium
- Propagation time (delay)
 - Time for a bit to traverse the link
- Processing time (delay)
 - time spent at the recipient or intermediate node for processing
- Queuing time (delay)
 - waiting time at the queue to be sent out

Model of Frame Transmission



(a) Error-free transmission

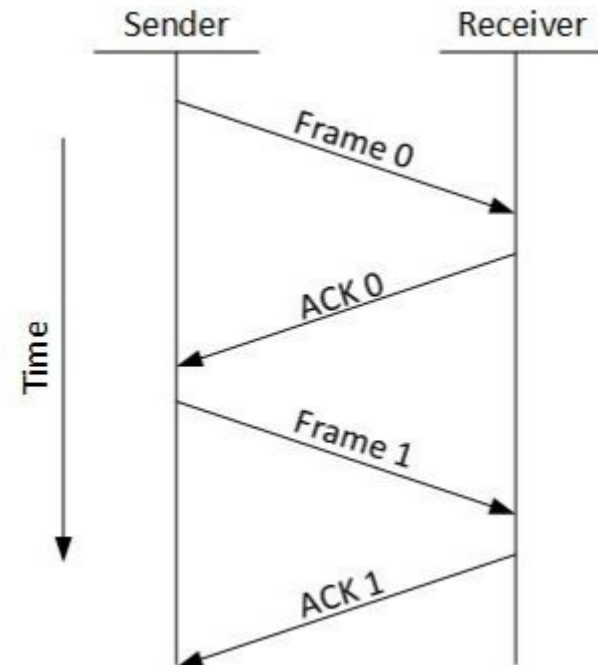
(b) Transmission with losses and errors



Stop-and- Wait Flow Control

- Stop-and –wait protocol is data link layer protocol for transmission of frames over **noiseless** channels
- It provides **unidirectional** data transmission with **flow control** facilities but with **out error control facilities**
- After transmitting one frame, the sender waits for an acknowledgement before transmitting the next frame.

- **Sender side**
 - **Rule:1** Send one data packet at a time
 - **Rule:2** Send the next packet only after receiving ACK for the previous
- **Receiver side**
 - **Rule:1** Receive and consume data packet
 - **Rule:2** After consuming packet, ACK need to be sent (Flow control)

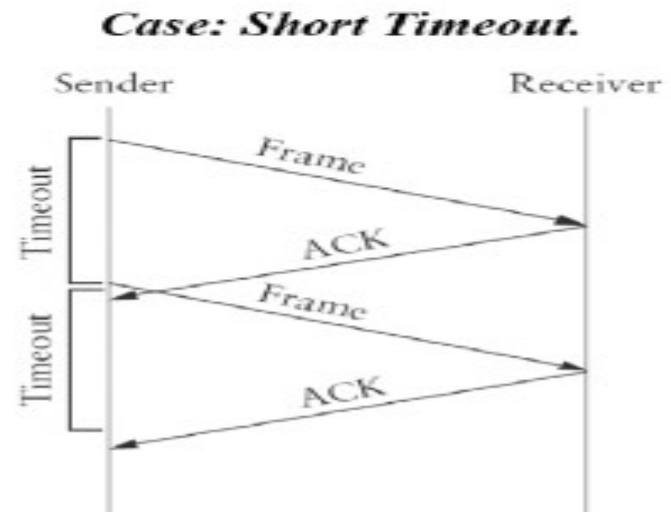
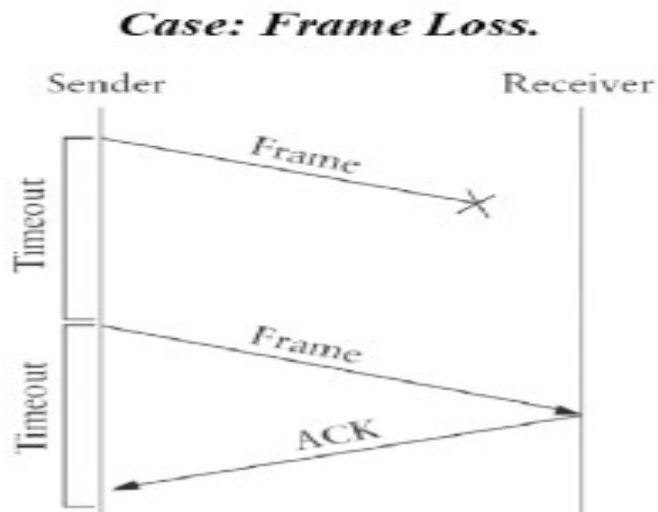
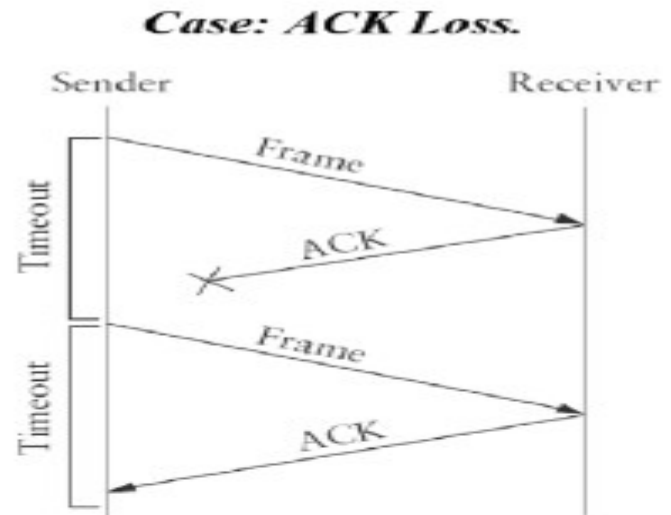
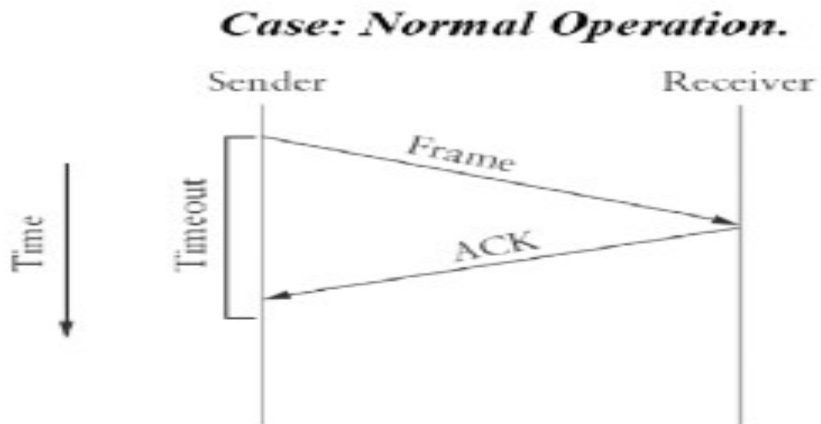


Problems of Stop-and-Wait Flow Control

- Problems of **data lost**
 - Sender waits for ACK for an infinite amount of time
 - Receiver waits for an infinite amount of time for the data
- Problems due to **lost ACK**
 - Sender waits for an infinite amount of time for ACK.
- Problem due to **delay ACK/data**
 - After timeout on the sender side, a delayed ACK might be wrongly considered as ACK of some other data packet.

Stop and Wait ARQ(Automatic Repeat Request)

- Idea of stop and wait protocol is straight forward
- After transmitting one frame the sender waits for an acknowledgement before transmitting the next frame.
- If the ACK does not arrive after a certain period of time, the sender **times out** and retransmits the original frame.
- It is a protocol for the noisy channel
- Stop-and-wait ARQ=Stop-and-Wait + Timeout Timer + Sequence number



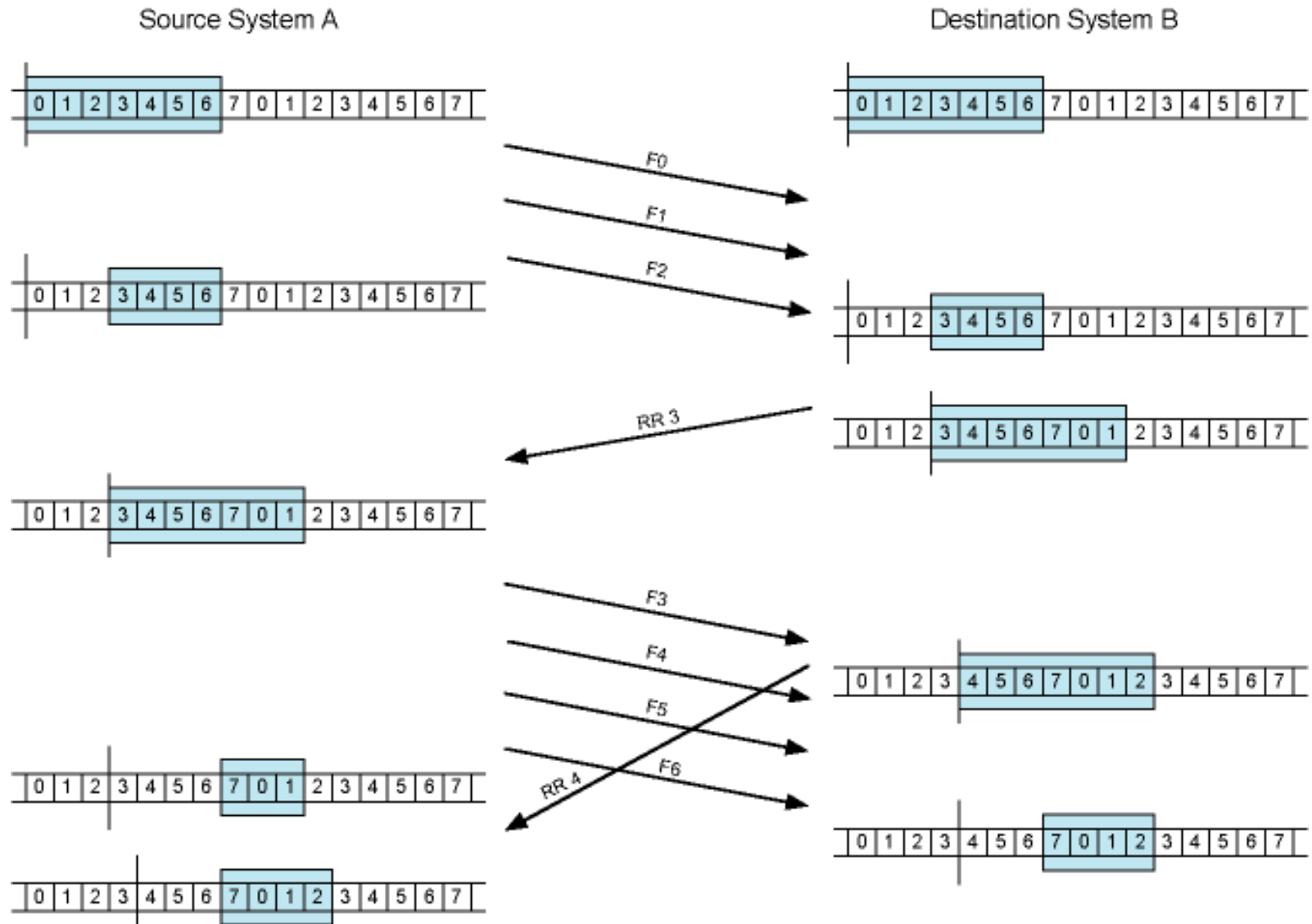
Stop and Wait ARQ drawback

- One frame at a time
- Poor utilization of bandwidth
- Poor performance

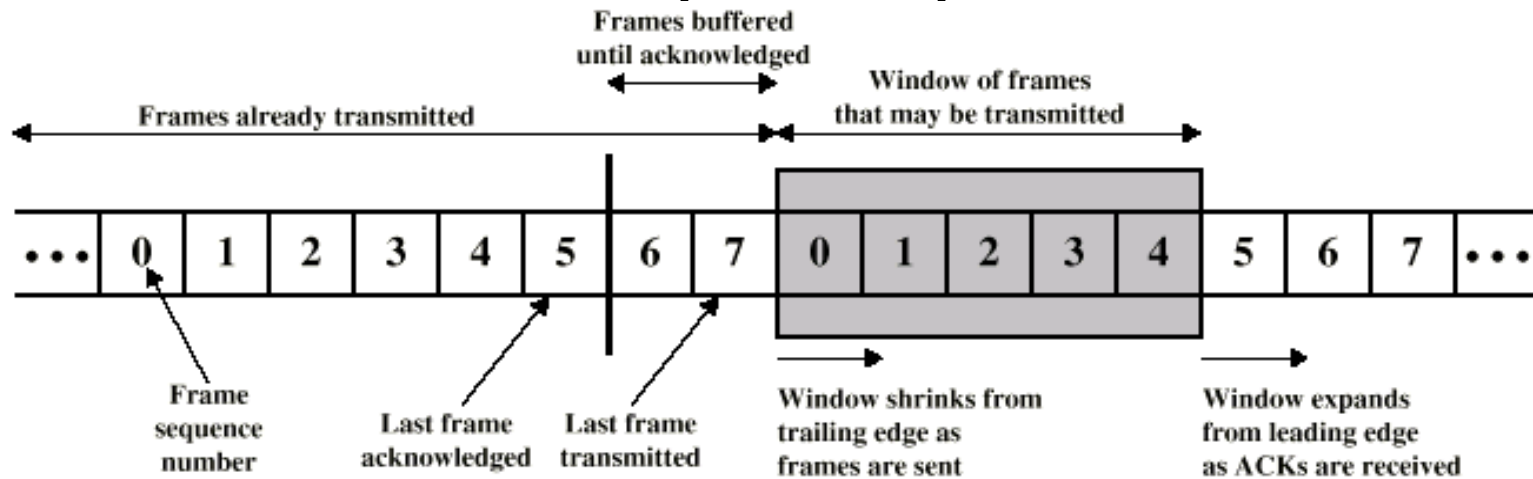
Sliding Window protocol

- Send multiple frames at a time
- Number of frames to be sent is based on **Window size**
- Each frame is numbered → sequence number

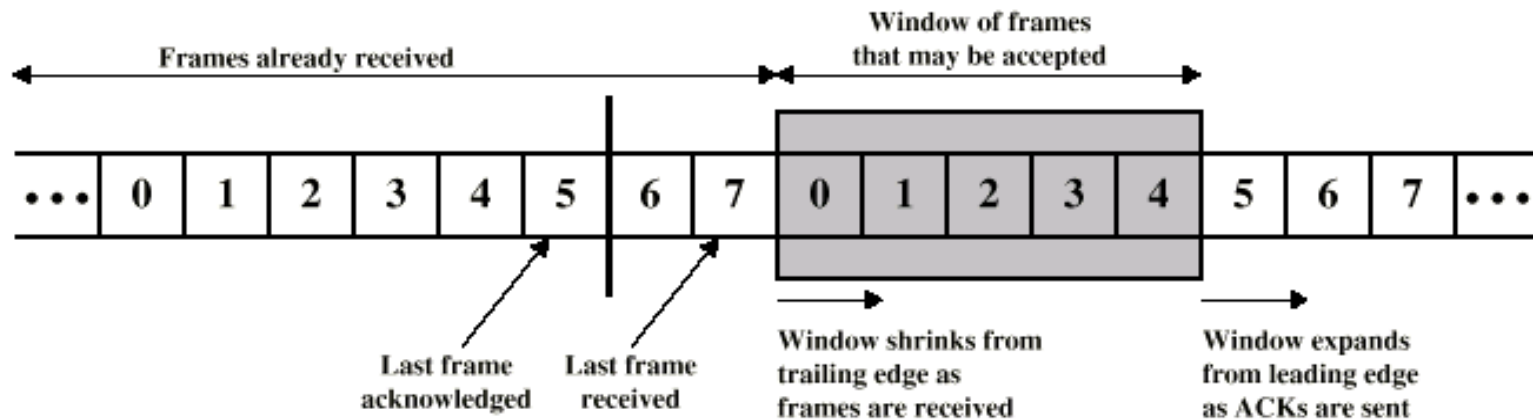
Example of a Sliding Window Protocol ($W = 7$)



Sliding Window Flow Control ($W = 7$)



(a) Sender's perspective



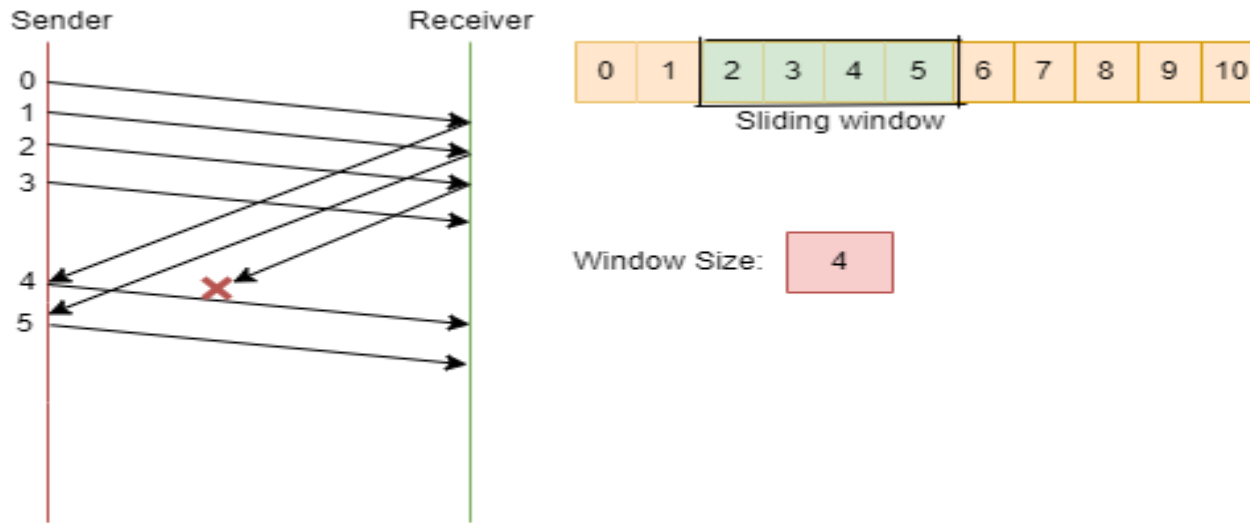
(b) Receiver's perspective

Go-Back-N ARQ

- Based on sliding window
- Go-Back-N ARQ uses the concept of protocol pipelining i.e. the sender can send multiple frames before receiving the acknowledgement for the first frame.
- There are finite number of frames and the frames are numbered in a sequential manner.
- The number of frames that can be sent depends on the window size of the sender.
 - N- the sender window size.

- If the acknowledgement of a frame is not received within an agreed upon time period, **all frames** in the current window are transmitted.

Working of Go-Back-ARQ protocol



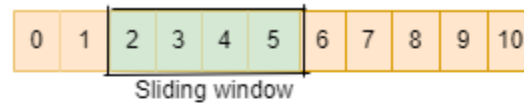
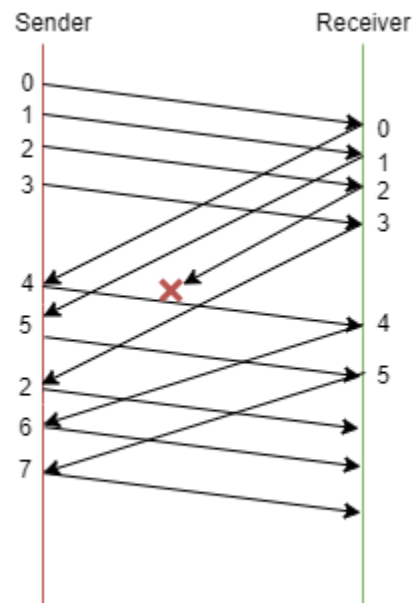
Home work

1. Station A needs to send a message consisting of 9 packets to station B using a sliding window (window size 3) and go-back-n- error control strategy. All packets are ready and immediately available for transmission. If every 5th packet that A transmits gets lost (but no ACKs from B ever get lost), then what is the number of packets that A will transmit for sending the message to B?

2. In Go back 4, if every 6th packet that is being transmitted is lost and if total number of packets to be sent is 10, then how many transmissions will be required?

Selective Repeat ARQ

- In selective Repeat ARQ, **only the erroneous or lost frames** are retransmitted, while correct frames are received and buffered.
- The receiver while keeping track of sequence numbers, buffers the frames in memory and sends NACK for only frame which is missing or damaged.
- The sender will send/retransmit packet for which NACK is received



Window Size: 4

- Minimizes retransmissions
- Receiver must maintain large enough buffer
- Complex implementation

Exercise

- Ex 1. In SR protocol frames through 0 to 4 have been transmitted. Now, imagine that 0 times out, 5(a new frame) is transmitted, 1 times out, 2 times out and 6(another new frame) is transmitted. At this point, what will be the outstanding packets in sender's window ?

- Host A wants to send 10 frames to Host B. the hosts agreed to go with SR ARQ. How many number of frames are transmitted by Host A if every 6th frame that is transmitted by host A is either corrupted or lost ? Also compare the number of transmissions of SRARQ with Go-Back-N ARQ.

ERROR DETECTION AND CORRECTION

Outcomes

- Upon completion of this chapter learner will be able to
 - Understand transmission errors
 - Know the type of errors
 - Understand error detection and correction
 - Know the various error detection techniques

Error control

Error control mechanisms are techniques (e.g., Checksum, CRC) that enable reliable delivery of data over unreliable communication channels

Error Control

Error

- data being transmitted over the network can be corrupted during transmission.
- The errors that are caused due to this transmission is known as **transmission error**.
- for reliable communication, errors must be detected and corrected.
- Error detection and correction are implemented either at the **data link layer** or at the **transport layer** of the OSI model

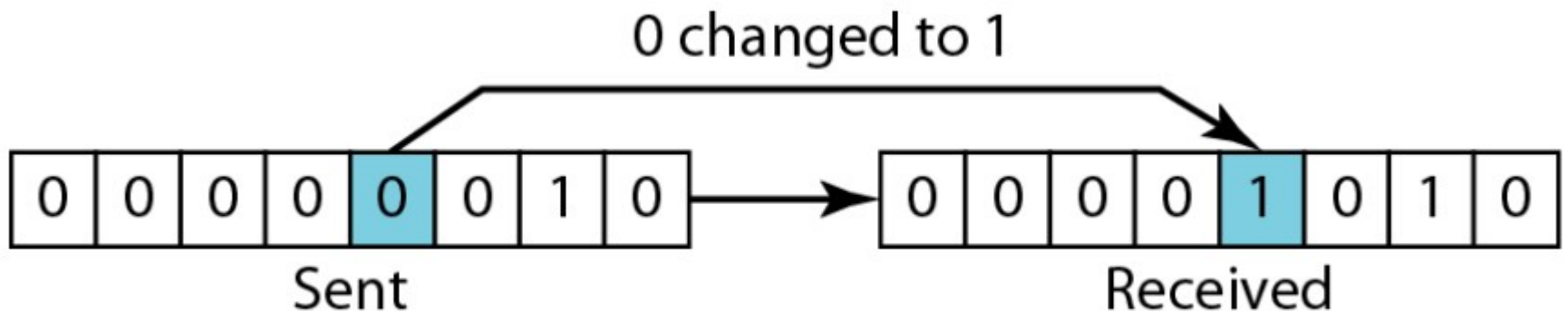
- So we have to
 - detect errors
 - if possible, correct errors
 - adopt flow control algorithms such that erroneous frames are retransmitted

Types of Errors

- Data-link layer uses some error control mechanism to ensure that frames (data bit streams) are transmitted with certain level of accuracy. But to understand how errors is controlled, it is essential to know what types of errors may occur.
- There are three types of errors
 - bit error
 - Burst error

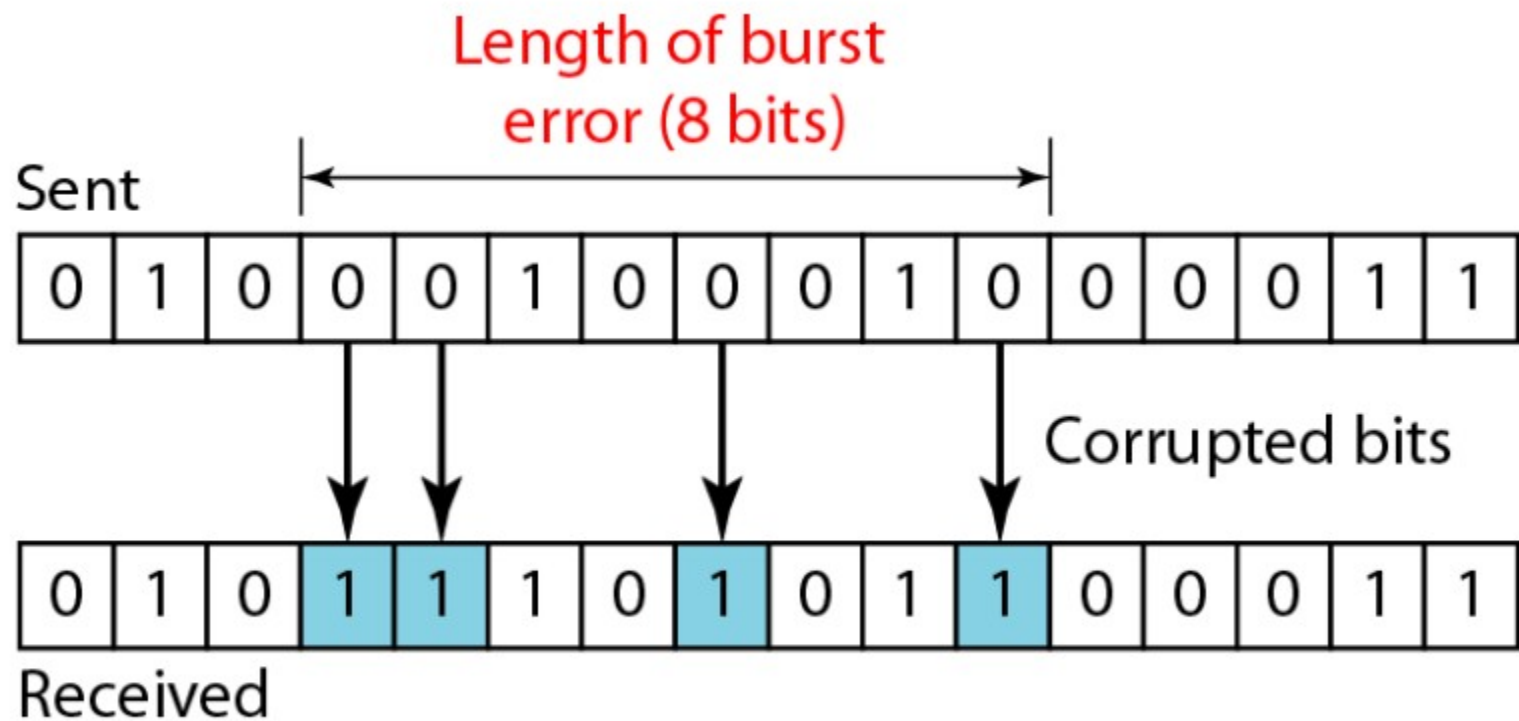
- **Single bit errors**

- a.k.a single bit error
- only 1 bit in the data unit has been changed
- not so common in real life



- **Burst errors**

- two or more bits in the data unit have changed
- most common case
- a burst error of length B is a contiguous sequence of B bits in which the first and the last and some intermediate bits are erroneously flipped.
 - not necessarily all bits between the first and the last one.



Error Detection and Correction

- **Error detection** refers to a class of techniques for detecting garbled messages
 - Adding some Extra bits to detect Occurrence of error
 - Not enough to detect the position of errors
- **Error correction**
 - Adding enough redundant bits to deduce what the correct bits are
 - Error Correction are too expensive and hard

How to detect the errors ?

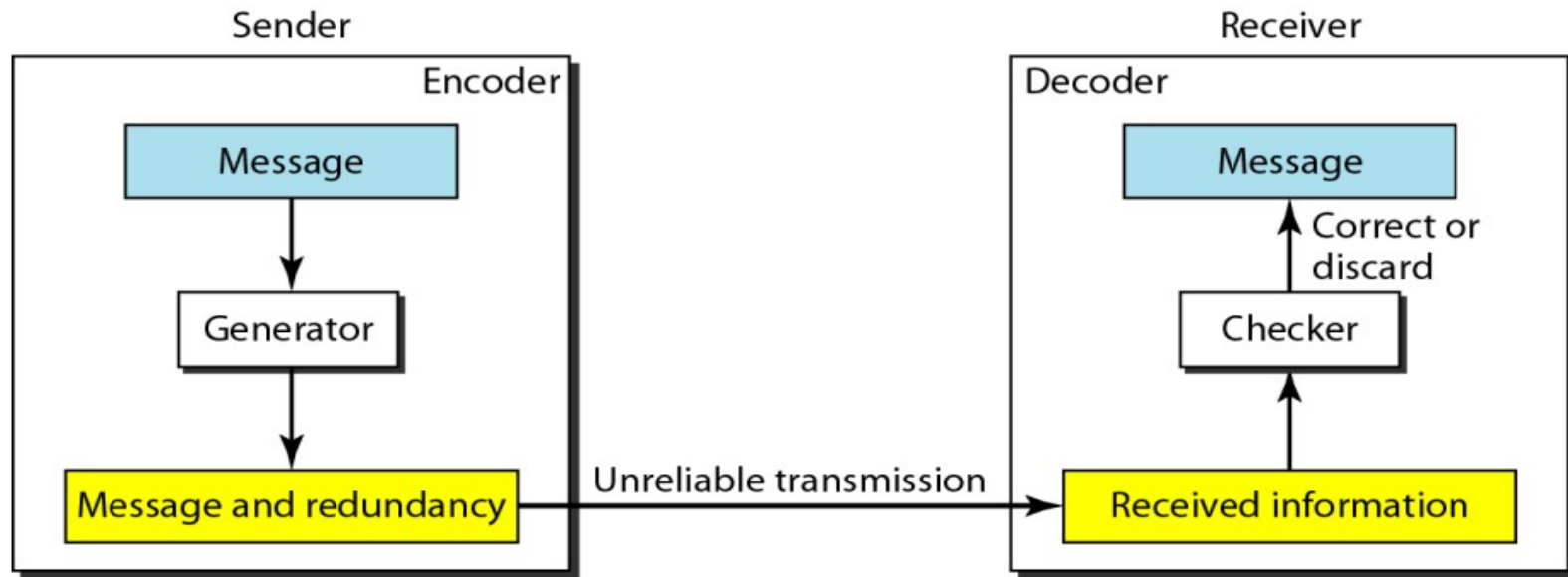
- **Redundancy**

- Error detection means to decide whether the received data is correct or not without having a copy of the original data.
- To detect or correct errors, we need to send some extra bits with the data . This extra bits are called **redundant bits**.
- These redundant bits are added by the sender and removed by the receiver.

Error correction

- Can be handled in two ways
 - Receiver can have the sender retransmit the entire data unit.
 - The receiver can use an error-correcting code, which automatically corrects certain errors

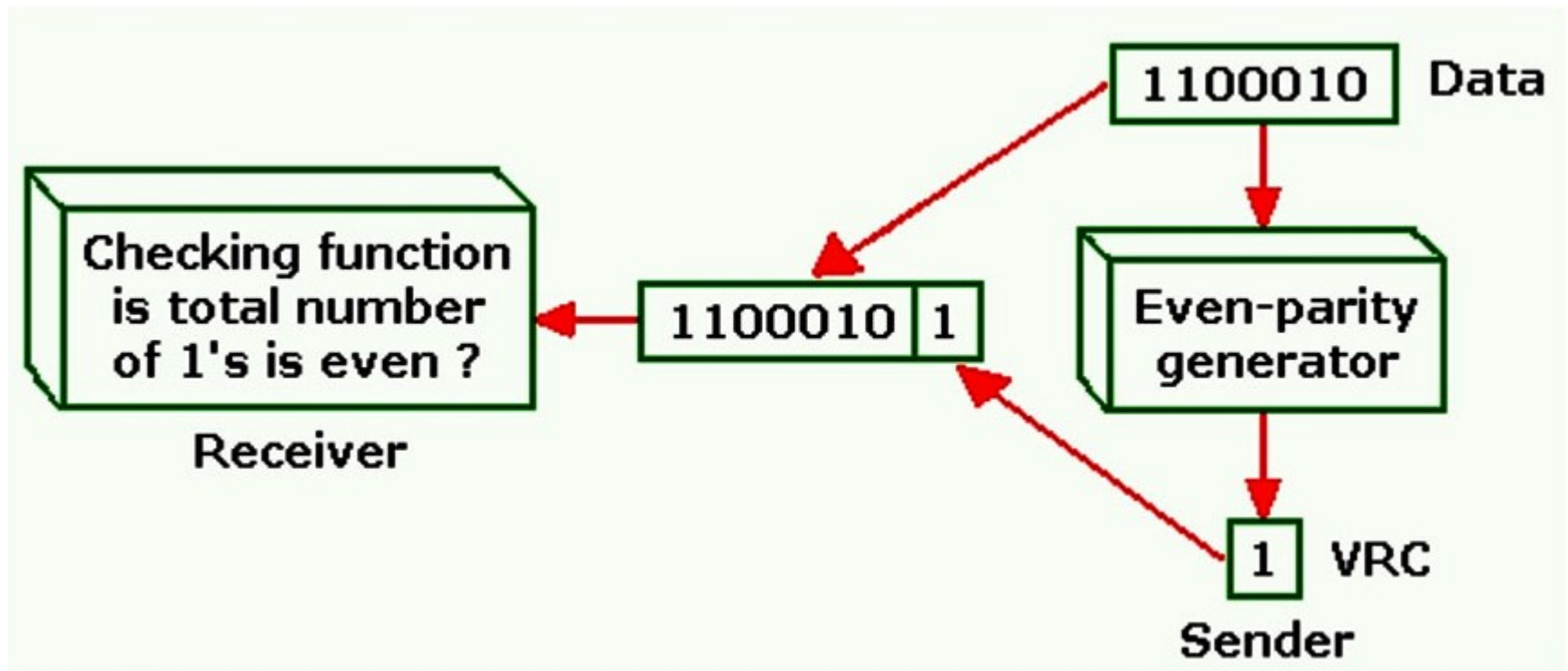
Error detection/correction



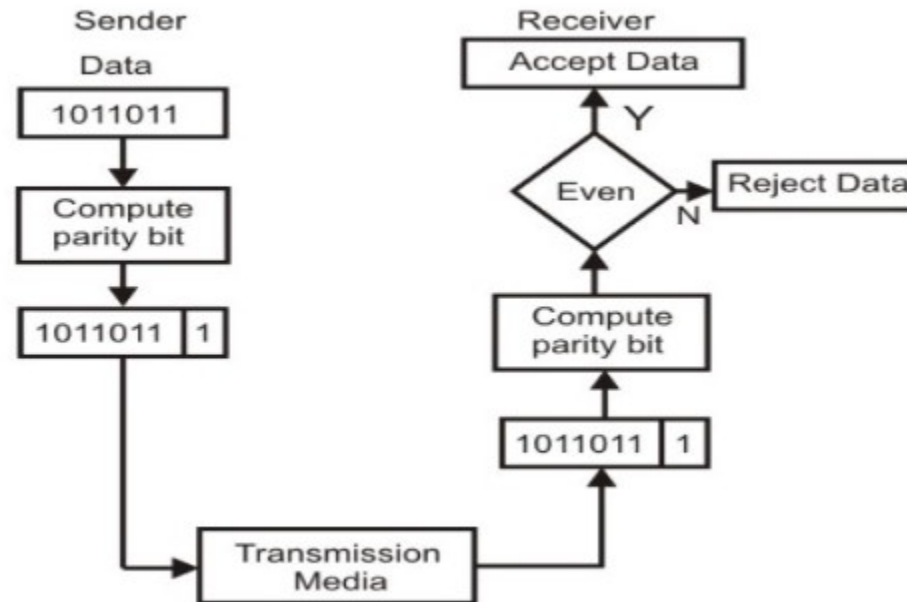
Error Detecting Codes techniques

- Simple Parity check or Vertical redundancy check (VRC)
- Two-dimensional Parity check or Longitudinal Redundancy Check(LRC)
- Cyclic redundancy check (CRC)
- Check Sum

Simple Parity Checking or One-dimension Parity Check



Even-parity checking scheme



Performance of Simple Parity Checks

- **can detect** all single bit errors.
- It can also detect **burst error only** if the number of bits in the error is **odd**.
- Sender:11100001→transmission error
10100001→ Receiver rejects the data
- Sender:11100001→transmission error
10100101→ Receiver accepts the data
 - Even number of bit errors goes undetected
 - thus not so useful

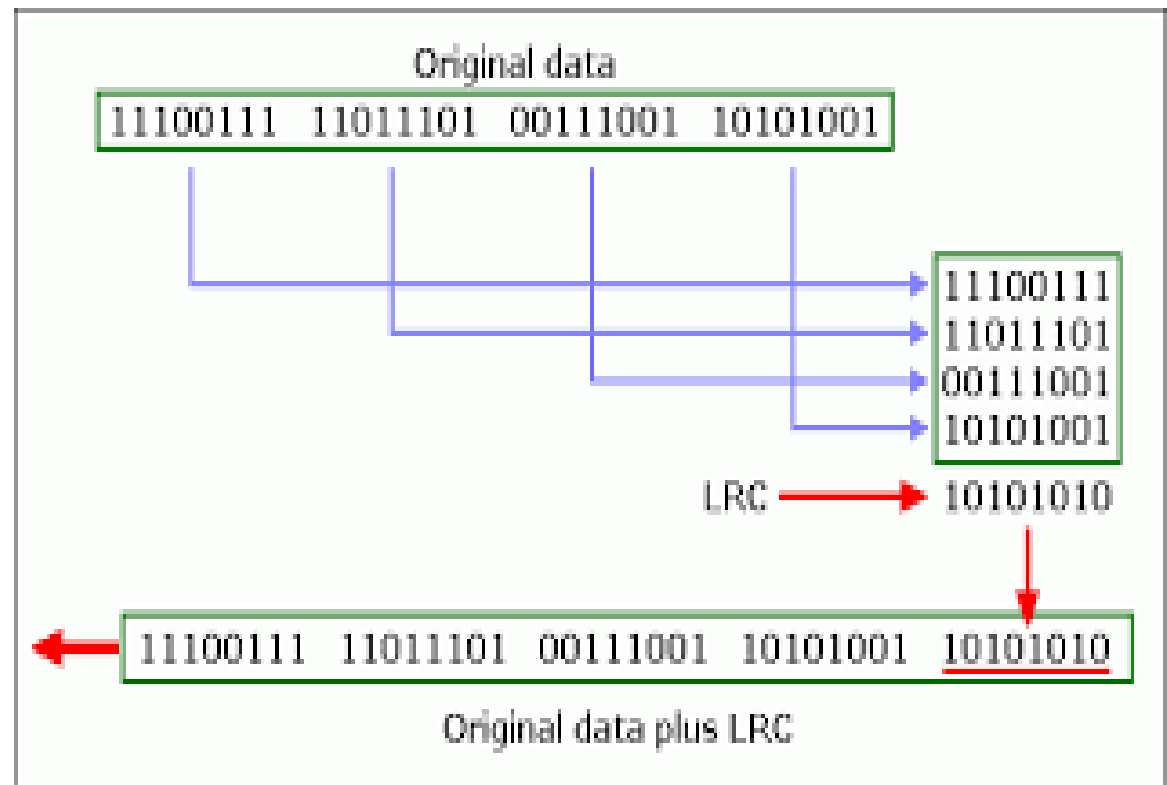
Longitudinal redundancy check(LRC)

- In LRC, a block of bits is organized in to rows and columns
- a.k.a Two dimensional parity
- The parity bit is calculated for each column and sent along with the data
- The block of parity acts as the redundant bits

LRC-Example

- Find the LRC for the data blocks 111000111
11011101 00111001 01010101

Odd no. 1s	1
Even no. 1 s	0



Two Dimensional Parity Check ...Performance

- improves error detection capability compared to simple parity check .
- Increases the likelihood of detecting burst errors.
- It can detect many burst errors, but not all.

- There is, however, one pattern of error that remains elusive. If two bits in one data unit are damaged and two bits in exactly same position in another data unit are also damaged, the LRC checker will not detect an error.

Example, if two data units: 11001100 and 10101100. If first and second from last bits in each of them is changed, making the data units as 01001110 and 00101110, the error cannot be detected by LRC Parity check.

- Home work
 - Find the LRC for the data blocks 01110111
10101001 01101001 10101010 and determine
the data that is transmitted ?

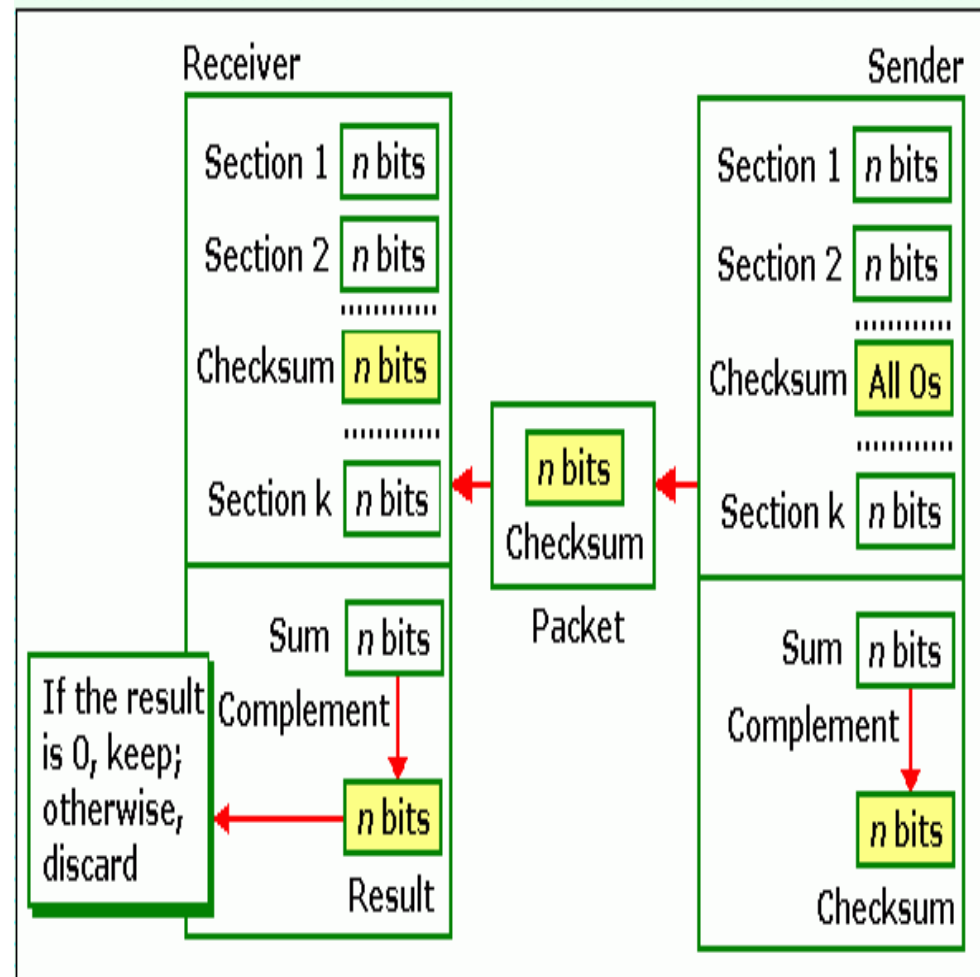
Checksum

- $\text{Checksum} = \text{Check} + \text{sum}$
- Sender side → Check sum creation
- Receiver side → Check sum validation

Checksum-operation at the sender side

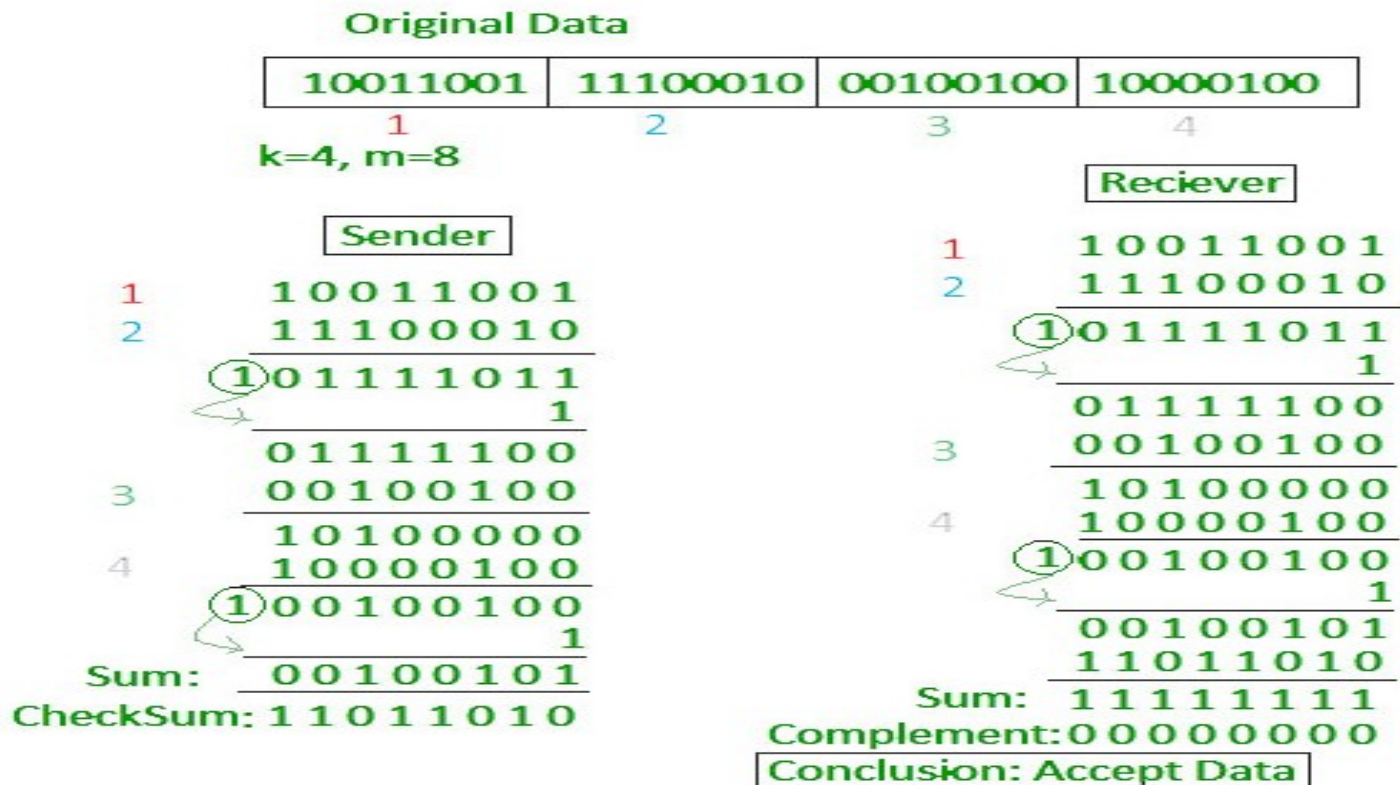
The sender's end:

- break the original message into 'k' number of blocks with 'n' bits in each block.
- Sum all the 'k' data blocks.
- Add the carry to the sum if any.
- Do the 1's complement to the sum = checksum
- The checksum segment is sent along with the data segments.



Checksum-example

- Consider the data unit to be transmitted is 10011001111000100010010010000100



- Collect all the data blocks including the checksum
- Sum the data blocks and checksum
- If the result is all 1's, accept; else, reject

Performance of Checksum

- Less overhead than two dimensional
- The checksum detects all errors involving an **odd** number of bits.
- At also detects most errors involving **even** number of bits.
- If one or more bits of a segment are damaged and the corresponding bit or bits of opposite value in a second segment are also damaged, the sum of those columns will not change and the receiver will not detect the error(S)

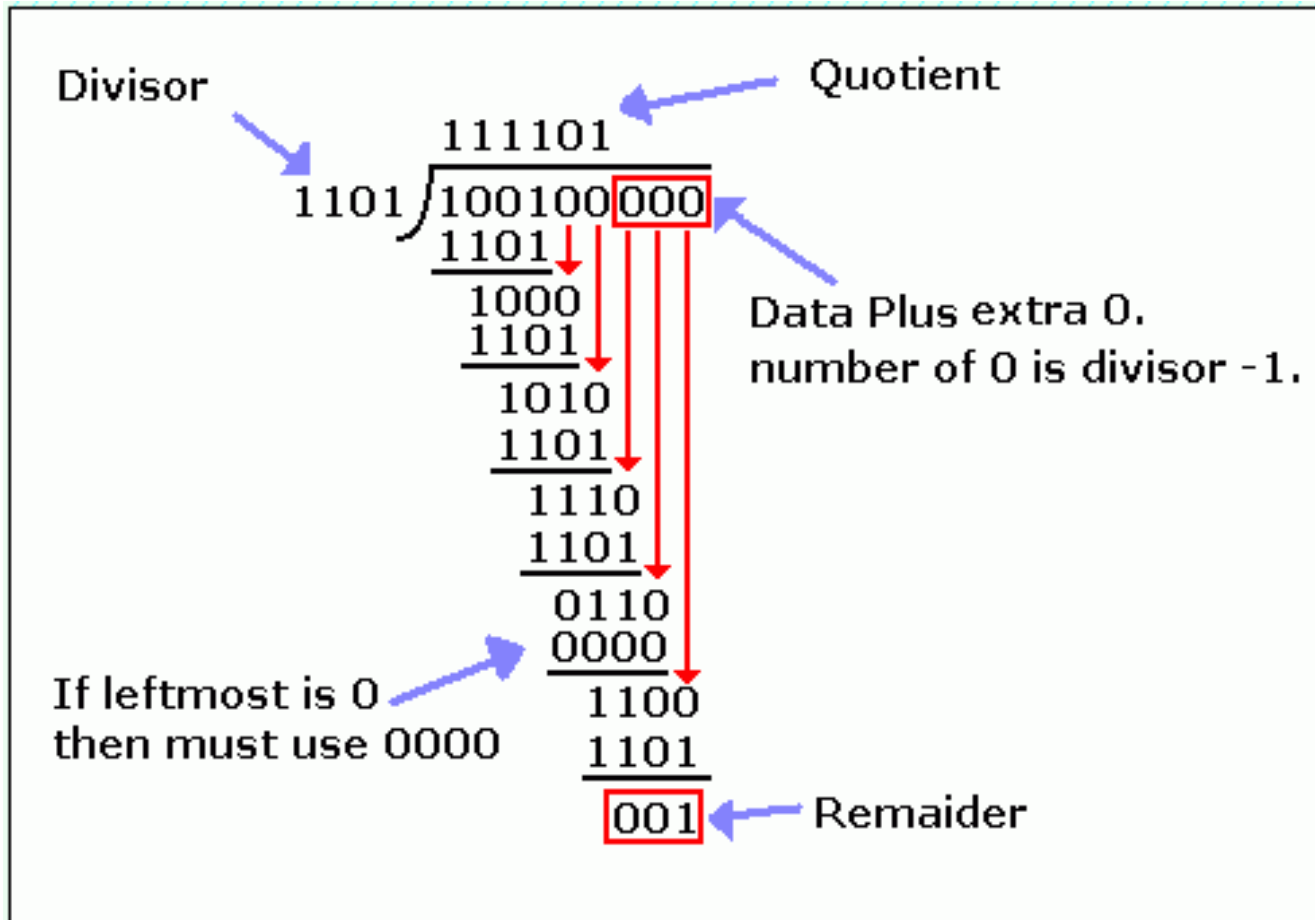
Cyclic Redundancy Check (CRC)

- One of the most powerful and commonly used error detecting codes.
- Find the CRC for the data blocks 100100 with the divisor 1101 ?

CRC generation at the sender side

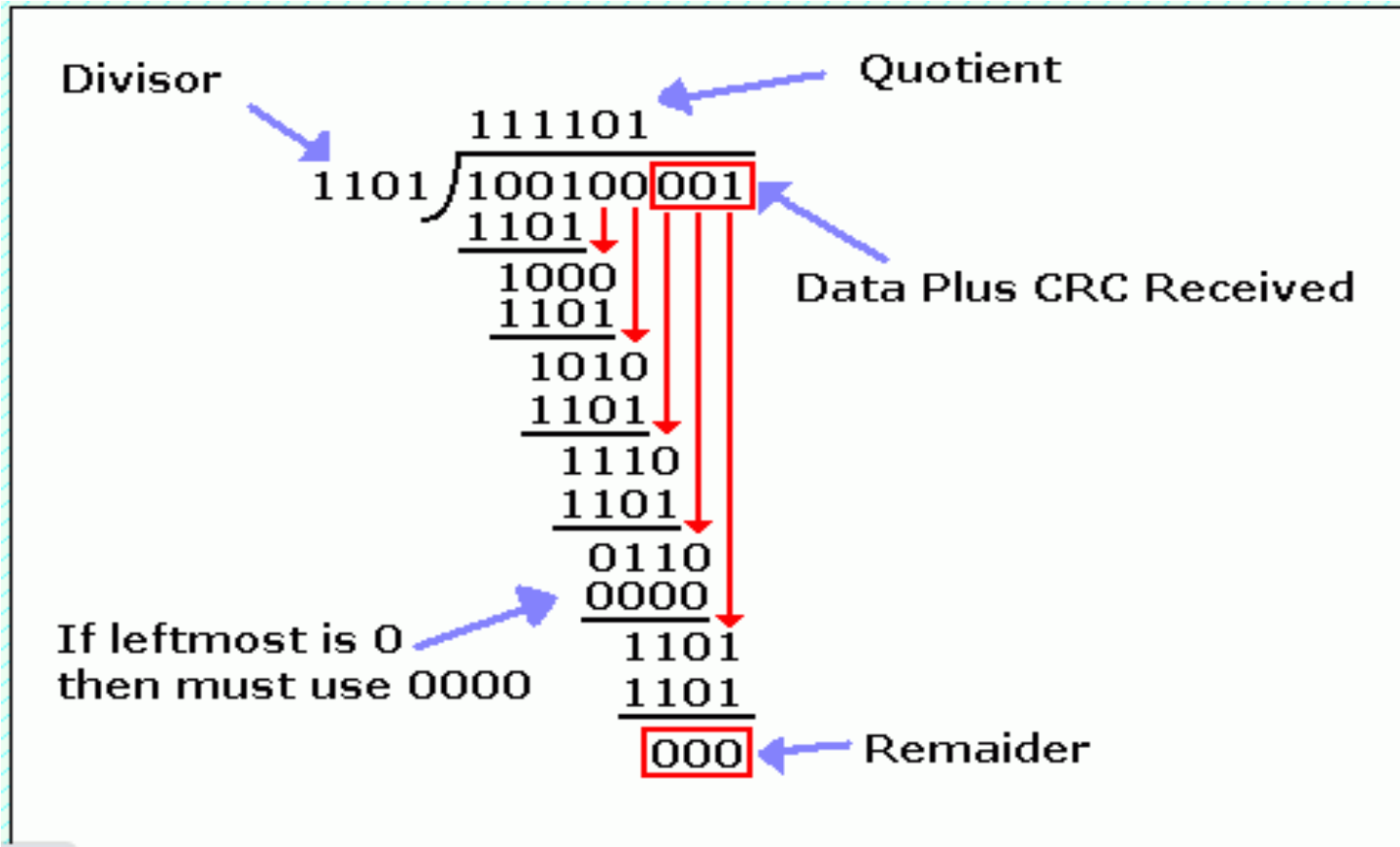
1. find the length of the divisor 'L'
2. Append "L-1" bits to the original message
3. Perform binary division operation
4. Remainder of the division → CRC

Sender side ?



Data Transmitted: 100100001

CRC-at the receiver side



Exercise

1. Find the CRC for 1110010101 with the divisor x^3+x^2+1 ?
2. 2. suppose we want to transmit the message 11001001 and protect it from errors using the CRC polynomial x^3+1 use polynomial long division to determine the message that should be transmitted. Corrupt the left-most third bit of the transmitted message and show that the error is created by the receiver using CRC technique

- Performance

- CRC can detect all single-bit errors
- CRC can detect all double-bit errors (three l's)
- CRC can detect any odd number of errors ($X+1$)
- CRC can detect all burst errors of less than the degree of the polynomial.
- CRC detects most of the larger burst errors with a high probability. .For example CRC-12 detects 99.97% of errors with a length 12 or more.

Error Correction

- Actions to be taken against
 - Lost frames
 - Damaged frames
- Error Correction codes are used to detect and repair mistakes that occur during data transmission from the transmitter to the receiver.
- Two Basic Approaches:
 - **Backward error correction:** When an error is detected in a frame, the sender is asked to retransmit the data/frame. This approach is known as Automatic Repeat Request (ARQ) technique.
 - **Forward error correction:** In this case, the receiver uses the error-correcting code which automatically corrects the errors.

- A single additional bit can detect the error, but cannot correct it.
- For correcting the errors, one has to know the **exact position** of the error. For example, If we want to calculate a single-bit error, the error correction code will determine which one of seven bits is in error. To achieve this, we have to add some additional redundant bits.

- Suppose **r** is the number of redundant bits and **d** is the total number of the data bits. The number of redundant bits **r** can be calculated by using the formula: $2^r \geq d + r + 1$
- The value of **r** is calculated by using the above formula. For example, if the value of **d** is 4, then the possible smallest value that satisfies the above relation would be 3.
- To determine the position of the bit which is in error, a technique developed by **R.W Hamming** is Hamming code which can be applied to any length of the data unit and uses the relationship between data units and redundant units.

Hamming Code

- **Data Bits:** the data you want to transmit
- **Parity bits:** The bit which is appended to the original data of binary bits so that the total number of 1s is even or odd.
- **Even parity:** To check for even parity, if the total number of 1s is even, then the value of the parity bit is 0. If the total number of 1s occurrences is odd, then the value of the parity bit is 1.
- **Odd Parity:** To check for odd parity, if the total number of 1s is even, then the value of parity bit is 1. If the total number of 1s is odd, then the value of parity bit is 0.

Algorithm of Hamming code

- An information of 'd' bits are added to the redundant bits 'p' to form d+p.
- The location of each of the (d+p) digits is assigned a decimal value.
- The 'p' parity bits are placed in positions 2^n where $n=0,1, 2, \dots, n$ positions. $\rightarrow 1,2, \dots 1,2, \dots 2^{r-1}$.
- At the receiving end, the parity bits are recalculated. The decimal value of the parity bits determines the position of an error.

Number of Data Bits (m)	Number of Redundancy Bits (k)	Total Bits (m+k)
1	2	3
2	3	5
3	3	6
4	3	7
5	4	9
6	4	10
7	4	11

- Let's understand the concept of Hamming code through an example:
- Suppose the original data is 1010 which is to be sent.

Total number of data bits 'd' = 4

Number of redundant bits r : $2^p \geq d+p+1$

$$2^p \geq 4+r+1$$

Therefore, the value of r is 3 that satisfies the above relation. **Total number of bits = $d+p = 4+3 = 7$;**

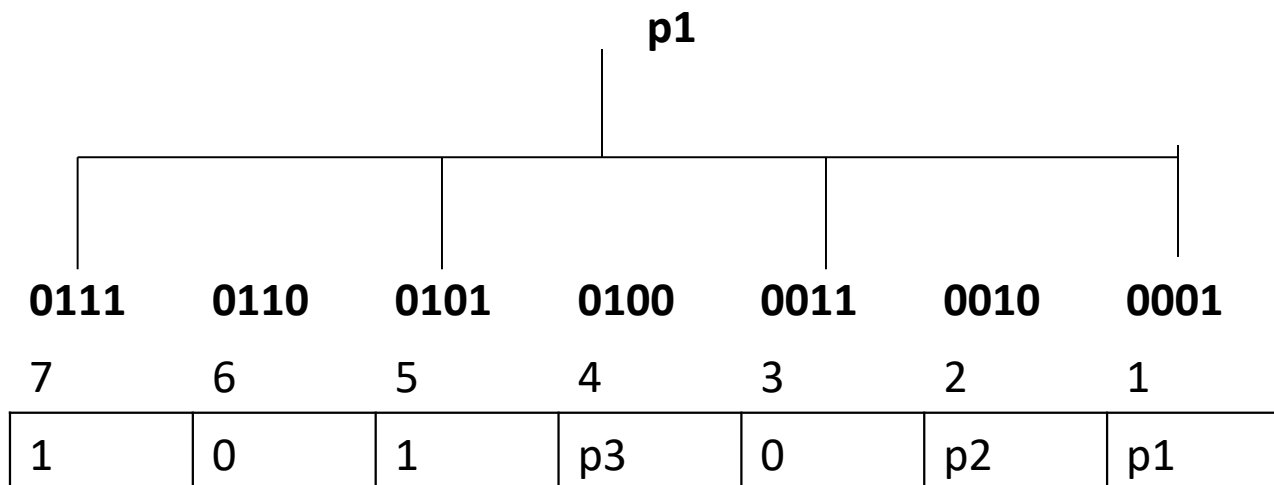
Determining the position of the redundant bits

- The number of redundant bits is 3. The three bits are represented by p_1 , p_2 , p_3 .
- The position of the redundant bits is calculated with corresponds to the raised power of 2. Therefore, their corresponding positions are 2^0 , 2^1 , 2^2 .
- The position of $p_1 = 1$
- The position of $p_2 = 2$
- The position of $p_3 = 4$

7	6	5	4	3	2	1
1	0	1	p_3	0	p_2	p_1

Determining the Parity bits

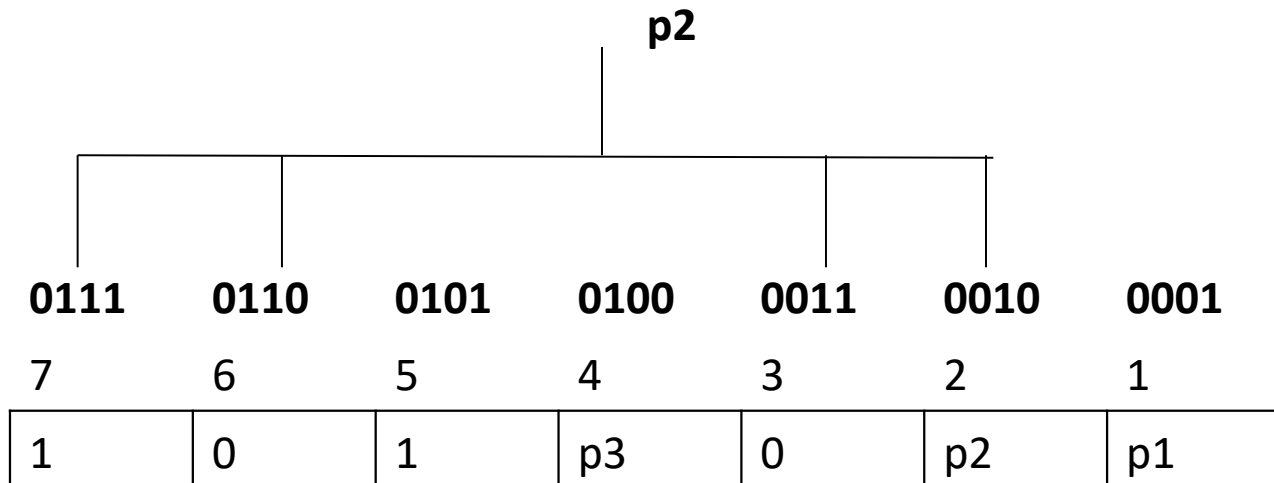
- Determining the p1 bit
- The p1 bit is calculated by performing a parity check on the bit positions whose binary representation includes 1 in the first position.



- We observe from the above figure that the bit positions that includes 1 in the first position are 1, 3, 5, 7.
- Now, we perform the even-parity check at these bit positions. The total number of 1 at these bit positions corresponding to p1 is **even, therefore, the value of the p1 bit is 0.**

Determining p2 bit

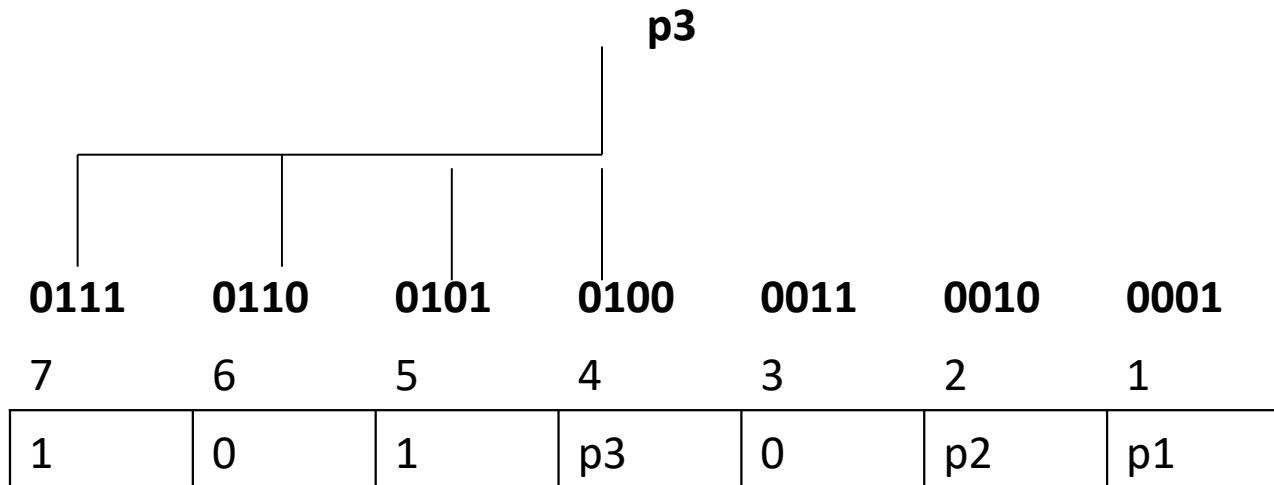
- The p2 bit is calculated by performing a parity check on the bit positions whose binary representation includes 1 in the second position.



- We observe from the above figure that the bit positions that includes 1 in the second position are **2, 3, 6, 7**.
- Now, we perform the even-parity check at these bit positions. The total number of 1 at these bit positions corresponding to p2 is **odd**, **therefore, the value of the p2 bit is 1**.

Determining p3 bit

- The p3 bit is calculated by performing a parity check on the bit positions whose binary representation includes 1 in the third position.



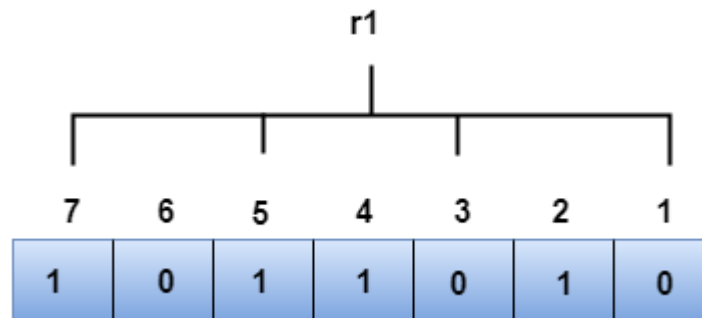
- We observe from the above figure that the bit positions that includes 1 in the third position are **4, 5, 6, 7**.
- Now, we perform the even-parity check at these bit positions. The total number of 1 at these bit positions corresponding to p3 is **even, therefore, the value of the p3 bit is 0.**

- **Data transferred is given below:**

7	6	5	4	3	2	1
1	0	1	0	0	1	0

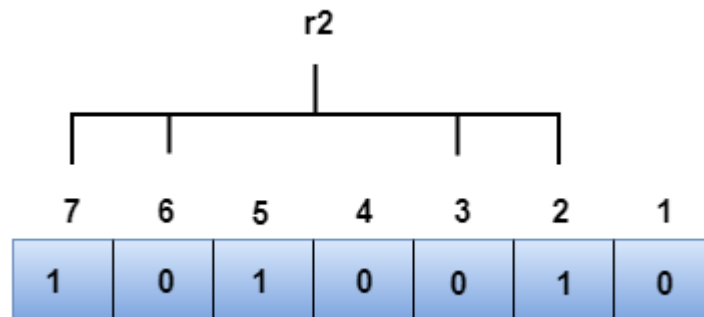
- Suppose the 4th bit is changed from 0 to 1 at the receiving end, then parity bits are recalculated.

- P1 bit
- The bit positions of the P1 bit are 1,3,5,7



- We observe from the above figure that the binary representation of r1 is 1100. Now, we perform the even-parity check, the total number of 1s appearing in the r1 bit is an even number. Therefore, the value of r1 is 0.

- P2 bit
- The bit positions of P2 bit are 2,3,6,7.



- We observe from the above figure that the binary representation of P2 is 1001. Now, we perform the even-parity check, the total number of 1s appearing in the r2 bit is an even number. Therefore, the value of r2 is 0.

- P3 bit
- The bit positions of P3 bit are 4,5,6,7.
- We observe from the above figure that the binary representation of P3 is 1011. Now, we perform the even-parity check, the total number of 1s appearing in the P4 bit is an odd number. Therefore, the value of P4 is 1.
- *The binary representation of redundant bits, i.e., P3P2P1 is 100, and its corresponding decimal value is 4. Therefore, the error occurs in a 4th bit position. The bit value must be changed from 1 to 0 to correct the error.*

Backward Error control

- Model of frame transmission
 - Data are sent as a sequence of frames.
 - Frames arrive at the same order as they are sent.
 - Each transmitted frame suffers an arbitrary and variable amount of delay before reception.
- In addition to above, following two types of errors may occur:
- Lost frame: A frame fails to arrive at the other side.
 - A noise burst may damage a frame to such an extent that it is not recognizable at the receiving end
- Damaged frame: A recognizable frame does arrive, but some of the bits are in error.

Backward Error Control

- Most common techniques for error control are based on some or all of the following:
 - Error detection: We have already discussed.
 - └ Positive acknowledgement: The destination returns a positive acknowledgement to successfully received errorfree frames.
 - └ Retransmission after timeout: The source retransmits a frame that has not been acknowledged after a predetermined amount of time.
 - └ Negative acknowledgement and retransmission: The destination returns a negative acknowledgement to frames in which an error is detected. The source retransmits such frames.

- Error Control Techniques Collectively, the mechanisms are referred to as Automatic Repeat Request (ARQ).
- Objective is to turn an unreliable data link into a reliable one.
- Three versions of ARQ are:
 - Stop-and-wait
 - Go-back-N Selective-repeat
 - Selective-reject (selective retransmission) ARQ